Convolutional Layers, Weight Sharing, Max Pooling, Stride, Zero Padding, and Hubel & Wiesel's Cortex Findings

Convolutional Neural Networks (CNNs) are a specialized type of deep neural network designed primarily for processing image data. Unlike traditional fully connected neural networks, CNNs exploit the spatial structure of images by learning local patterns such as edges, corners, textures, and shapes. This makes them highly effective for computer vision tasks including image classification, object detection, facial recognition, and medical image analysis. CNNs achieve this efficiency through convolutional layers, weight sharing, pooling operations, stride, and padding.

Convolutional Layers

The convolutional layer is the fundamental building block of a CNN. Its primary purpose is to automatically detect important visual features from an input image. Rather than connecting every input pixel to every neuron, a convolutional layer applies a small matrix called a **filter** or **kernel** across the image. Typical filter sizes are 3×3, 5×5, or 7×7.

As the filter slides over the image, it performs element-wise multiplication between the filter values and the corresponding image pixels. The products are summed together to produce a single value in the output feature map. This process is repeated for every possible position of the filter, allowing the network to detect where certain patterns appear in the image.

Different filters learn different features during training. For example:

- One filter may detect horizontal edges.
- Another may detect vertical edges.
- Others may detect textures, curves, or colour transitions.

In deeper CNN layers, these simple features are combined into more complex structures such as eyes, wheels, faces, or complete objects. Thus, early layers learn low-level features while deeper layers learn high-level semantic representations.

Weight Sharing

One of the key advantages of convolutional layers is **weight sharing**. Instead of learning separate weights for every image location, the same filter weights are reused across the entire image.

For example, a single 3×3 filter always contains only **9 learnable weights**, regardless of whether the image is 32×32 or 1024×1024 pixels. As the filter moves across different regions, those same weights are applied repeatedly.

Weight sharing provides several important benefits:

- **Dramatically reduces the number of parameters**, making CNNs much more memory-efficient than fully connected networks.
- **Reduces overfitting** because there are fewer parameters to learn.
- **Allows translation invariance**, meaning a feature can be recognised regardless of where it appears in the image.
- **Improves computational efficiency**, enabling faster training and inference.

Without weight sharing, every filter position would require an entirely new set of parameters, making CNNs computationally impractical for large images.

Max Pooling

After one or more convolutional layers, CNNs commonly apply a **pooling layer**, whose purpose is to reduce the spatial dimensions of feature maps while retaining the most important information.

The most common pooling operation is **max pooling**. In max pooling, a small window (usually 2×2) moves across the feature map and outputs only the largest value within each window.

For example,

Input:

1 4

3 2

Output = 4

The maximum activation is preserved because it represents the strongest detected feature within that region.

Max pooling provides several advantages:

- Reduces image size and computational cost.
- Decreases memory usage.
- Makes the network more robust to small translations and distortions.
- Helps reduce overfitting by simplifying feature representations.

A 2×2 max pooling layer with stride 2 reduces both the height and width by approximately half. For example, a 28×28 feature map becomes 14×14 after pooling.

Stride

Stride determines how many pixels the filter moves each time it slides across the image.

- **Stride = 1:** the filter moves one pixel at a time.
- **Stride = 2:** the filter jumps two pixels at a time.
- Larger stride values continue skipping more positions.

A larger stride reduces the output feature map size because fewer filter positions are evaluated.

For an input of size **N**, filter size **F**, padding **P**, and stride **S**, the output dimension is:

$$\text{Output Size} = \frac{N - F + 2P}{S} + 1$$

(assuming the result is an integer).

For example:

- Input = 32×32
- Filter = 3×3
- Padding = 0
- Stride = 1

Output:

(32 − 3)/1 + 1 = **30**

Resulting feature map = **30×30**

If the stride increases to 2:

(32 − 3)/2 + 1 = **15.5**, which in practice gives **15×15** when only valid filter positions are used.

Thus, increasing stride decreases computational cost but also reduces spatial resolution. Choosing stride involves balancing efficiency with preservation of image detail.

Zero Padding

Without padding, every convolution slightly shrinks the image because the filter cannot extend beyond the image boundaries. After many convolutional layers, the feature maps become very small.

Zero padding solves this problem by surrounding the image with extra pixels whose values are zero before applying convolution.

For example, adding one layer of zero padding to a 5×5 image produces a 7×7 padded image.

Benefits of zero padding include:

- Preserves spatial dimensions.
- Prevents excessive shrinking after multiple convolutions.
- Allows edge pixels to contribute equally during feature extraction.
- Enables deeper CNN architectures without rapidly losing spatial information.

With padding equal to 1 and a 3×3 filter using stride 1, the output size remains the same as the input size. This is often called **same padding**.

Without padding, the output becomes smaller after every convolution, known as **valid convolution**.

Hubel & Wiesel's Cortex Findings

The design of convolutional neural networks was strongly inspired by the pioneering neuroscience research of **David Hubel and Torsten Wiesel**. While studying the visual cortex of cats, they discovered that certain neurons respond only to specific visual patterns such as edges, lines, orientations, and movement within small regions of the visual field. They also found that increasingly complex neurons combine these simple features into higher-level representations. CNNs mimic this hierarchical organisation by using convolutional layers to detect simple local features first and progressively combining them into complex object representations in deeper layers.